Experiment 1: Solving Water Jug and Missionaries & Cannibals Problems

Aim / Objective

- To solve the two-water jug problem by manually accepting user-defined moves.
- To understand classic Artificial Intelligence problem-solving techniques using constraints.
- To implement the Missionaries and Cannibals problem using Python.

Theory

The Water Jug problem is a known challenge that shows how to solve problems using actions that are allowed. We have two jugs that can hold an amount of water. The Water Jug problem is to measure an amount of water using these two jugs. To solve the Water Jug problem, we do things like fill the jugs empty them or pour water from one jug to the other. Each time we do something it changes the state of the Water Jug problem. We keep doing things and changing the state of the Water Jug problem until we find the solution, to the Water Jug problem by following the rules. The answer to this experiment is derived by manually inputting moves, aiding in the comprehension of how various actions influence the system state and yield the intended outcome.

The Missionaries and Cannibals problem is a classic AI puzzle involving three missionaries and three cannibals who must cross a river using a boat that can carry at most two people. The main constraint is that missionaries cannot be outnumbered by cannibals on either bank at any time.

Algorithm / Methodology

1. Water Jug Problem

1. Define the capacities of the two water jugs.
2. Initialize both jugs to empty state.
3. Accept the goal amount from the user.
4. Display available operations to the user.
5. Perform the selected operation on the jugs.

6. Display the current state after each move.
7. Repeat the process until the goal state is reached.

2. Missionaries & Cannibals Problem

1. Represent the state as (leftBank, rightBank, boat_position).
2. Define valid moves based on boat capacity: (1,0), (0,1), (2,0), (0,2), (1,1).
3. Check constraints: missionaries should never be outnumbered by cannibals on any bank.
4. Track visited states to avoid cycles.
5. Backtrack if no valid moves are available.
6. Repeat until all missionaries and cannibals reach the opposite bank.

Dataset Description

No external dataset is used in this experiment. The inputs consist of water jug capacities, goal amount, and user-selected operations provided during program execution.

Implementation

1.Water Jug Problem

```
# Allowed Operations

def fillFromTap(jug, capacity):

  jug = capacity

  return jug;


def fillFromJug(from_jug, to_jug, capacityOfJug):

  if(from_jug + to_jug > capacityOfJug):

    return (capacityOfJug, from_jug - (capacityOfJug - to_jug))

  else:

    return (from_jug + to_jug, 0)
```

```
def empty(jug):
  jug = 0
  return jug;
# Input
x = 0
y = 0
capacity_x = int(input("Enter capacity of x: "))
capacity_y = int(input("Enter capacity of y: "))
print("1. Exactly l Litres\t2. Fill l Litres")
problem_type = int(input("Enter no. of goal type :"))
goal = []
if(problem_type == 1):
  l = int(input("Enter Goal: "))
  if(capacity_x >= l):
    goal.append([l,0])
    if(capacity_y >= l):
      goal.append([0,l])
  elif(capacity_y >= l):
    goal.append([-1,-1])
    goal.append([0,l])
  if(capacity_x > l and capacity_y > l):
    print("Invalid Goal")
  print(goal)
elif(problem_type == 2):
  l = int(input("Enter Goal: "))
  if(capacity_x >= l or capacity_y >= l):
    goal.append(l)
```

```
    print(goal[0])
  else:
    print("Invalid Goal")
else:
  print("Invalid Goal Type")
  exit()


# User Input To solve Problem
count = 0
if(problem_type == 1):
  while((x != goal[0][0] or y != goal[0][1]) and (x != goal[1][0] or y !=
goal[1][1])):
    count += 1
    print(f"\nCurrent State: x = {x} and y = {y}\n")
    print("1. Fill x from Tap")
    print("2. Fill y from Tap")
    print("3. Fill x from y")
    print("4. Fill y from x")
    print("5. Empty x")
    print("6. Empty y")
    choice = int(input("Enter your choice: "))
    if(choice == 1):
      x = fillFromTap(x, capacity_x)
    elif(choice == 2):
      y = fillFromTap(y, capacity_y)
    elif(choice == 3):
      x, y = fillFromJug(x, y, capacity_x)
    elif(choice == 4):
```

```
      y,x = fillFromJug(y, x, capacity_y)
    elif(choice == 5):
      x = empty(x)
    elif(choice == 6):
      y = empty(y)
    else:
      print("Invalid choice")
  print(f"Current State: x = {x} and y = {y}\n")
  print(f"No. of Operations required: {count}")
else:
  while(x != goal[0] and y != goal[0]):
    count += 1
    print(f"Current State: x = {x} and y = {y}\n")
    print("1. Fill x from Tap")
    print("2. Fill y from Tap")
    print("3. Fill x from y")
    print("4. Fill y from x")
    print("5. Empty x")
    print("6. Empty y")
    choice = int(input("Enter your choice: "))
    if(choice == 1):
      x = fillFromTap(x, capacity_x)
    elif(choice == 2):
      y = fillFromTap(y, capacity_y)
    elif(choice == 3):
      x, y = fillFromJug(x, y, capacity_x)
    elif(choice == 4):
      y,x = fillFromJug(y, x, capacity_y)
```

```
    elif(choice == 5):
      x = empty(x)
    elif(choice == 6):
      y = empty(y)
    else:
      print("Invalid choice")
  print(f"\nCurrent State: x = {x} and y = {y}")
  print(f"No. of Operations required: {count}")
```

2. Missionaries & Cannibals Problem

```
def getValidOperations(leftBank, rightBank, direction):
  valid_operations = []
  possible_moves = [(1, 0), (0, 1), (2, 0), (0, 2), (1, 1)]

  if(direction == 'left'):
    current_bank = rightBank
  else:
    current_bank = leftBank

  for c_move, m_move in possible_moves:
    if current_bank['cannibals'] >= c_move and current_bank['missionaries']
>= m_move:
      remaining_c = current_bank['cannibals'] - c_move
      remaining_m = current_bank['missionaries'] - m_move
      if isValidState(remaining_c, remaining_m):
        if c_move + m_move >= 1:
          valid_operations.append((c_move, m_move))

  return valid_operations
```

```
def leftToRight(leftBank, rightBank, visited_states):
  valid_operations = getValidOperations(leftBank, rightBank, 'right')


  for c_move, m_move in valid_operations:
      new_leftBank, new_rightBank, new_direction = applyMove(leftBank, rightBank, 'right', c_move, m_move)
    if isValidState(new_leftBank['cannibals'], new_leftBank['missionaries']) and \
                                    isValidState(new_rightBank['cannibals'], new_rightBank['missionaries']):
      new_state_tuple = (
          tuple(sorted(new_leftBank.items())),
          tuple(sorted(new_rightBank.items())),
          new_direction
      )
      if new_state_tuple not in visited_states:
        visited_states.add(new_state_tuple)
        operation = f"Move {c_move} Cannibals and {m_move} Missionaries from Left to Right"
        return new_leftBank, new_rightBank, new_direction, operation, True
  return leftBank, rightBank, 'right', None, False


def rightToLeft(leftBank, rightBank, visited_states):
  valid_operations = getValidOperations(leftBank, rightBank, 'left')


  for c_move, m_move in valid_operations:
      new_leftBank, new_rightBank, new_direction = applyMove(leftBank, rightBank, 'left', c_move, m_move)
    if isValidState(new_leftBank['cannibals'], new_leftBank['missionaries']) and \
```

```
                                    isValidState(new_rightBank['cannibals'],
new_rightBank['missionaries']):
      new_state_tuple = (
          tuple(sorted(new_leftBank.items())),
          tuple(sorted(new_rightBank.items())),
          new_direction
      )
      if new_state_tuple not in visited_states:
        visited_states.add(new_state_tuple)
        operation = f"Move {c_move} Cannibal and {m_move} Missionaries from
Right to Left"
        return new_leftBank, new_rightBank, new_direction, operation, True
  return leftBank, rightBank, 'left', None, False


def AI():
  solution = []
  visited_states = set()
  path_states = []

  leftBank = {'cannibals': 0, 'missionaries': 0}
  rightBank = {'cannibals': 3, 'missionaries': 3}
  direction = 'left'

  # Initial state
  initial_state_tuple = (
      tuple(sorted(leftBank.items())),
      tuple(sorted(rightBank.items())),
      direction
  )
```

```
  visited_states.add(initial_state_tuple)

  path_states.append((leftBank.copy(), rightBank.copy(), direction))

  max_iterations = 1000

  iterations = 0


  while (leftBank['cannibals'] != 3 or leftBank['missionaries'] != 3) and
path_states and iterations < max_iterations:

    iterations += 1

    current_leftBank, current_rightBank, current_direction = path_states[-
1]

    leftBank = current_leftBank.copy()

    rightBank = current_rightBank.copy()

    direction = current_direction


    new_leftBank = {}

    new_rightBank = {}

    new_direction = ''

    operation = None

    isSolvable = False


    if direction == 'left':

       new_leftBank, new_rightBank, new_direction, operation, isSolvable =
rightToLeft(leftBank, rightBank, visited_states)

    else:

       new_leftBank, new_rightBank, new_direction, operation, isSolvable =
leftToRight(leftBank, rightBank, visited_states)


    if isSolvable:

      solution.append(operation)
```

```
            path_states.append((new_leftBank.copy(),   new_rightBank.copy(),
new_direction))
      leftBank = new_leftBank
      rightBank = new_rightBank
      direction = new_direction
    else:
      if len(path_states) > 1:
        path_states.pop()
      if solution:
        solution.pop()
      if path_states:
          leftBank, rightBank, direction = path_states[-1]
      else:
          break

  if leftBank['cannibals'] == 3 and leftBank['missionaries'] == 3:
    print("Solution Found:")
    for step in solution:
      print(step)
  else:
    print("No solution found.")

AI()
```

Output / Result

1. Water Jug Problem

```
Current State: x = 0 and y = 0

1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 2

Current State: x = 0 and y = 5

1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 3

Current State: x = 3 and y = 2

1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 5

Current State: x = 0 and y = 2
```

```
1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 3

Current State: x = 2 and y = 0

1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 2

Current State: x = 2 and y = 5

1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 3

Current State: x = 3 and y = 4
```

```
1. Fill x from Tap
2. Fill y from Tap
3. Fill x from y
4. Fill y from x
5. Empty x
6. Empty y
Enter your choice: 5
Current State: x = 0 and y = 4

No. of Operations required: 7
```

2. Missionaries & Cannibals Problem

```
Solution Found:
Move 2 Cannibal and 0 Missionaries from Right to Left
Move 1 Cannibals and 0 Missionaries from Left to Right
Move 2 Cannibal and 0 Missionaries from Right to Left
Move 1 Cannibals and 0 Missionaries from Left to Right
Move 0 Cannibal and 2 Missionaries from Right to Left
Move 1 Cannibals and 1 Missionaries from Left to Right
Move 0 Cannibal and 2 Missionaries from Right to Left
Move 1 Cannibals and 0 Missionaries from Left to Right
Move 2 Cannibal and 0 Missionaries from Right to Left
Move 1 Cannibals and 0 Missionaries from Left to Right
Move 2 Cannibal and 0 Missionaries from Right to Left
```

Result Analysis

The Water Jug problem reaches the target amount when valid operations are performed in the correct sequence. Manual input helps visualize state transitions clearly. The experiment confirms correct implementation of state-space traversal and logical decision-making.

The program successfully transfers all missionaries and cannibals to the opposite bank without violating constraints. The solution sequence shows correct state validation and backtracking. This demonstrates proper AI problem-solving using systematic state exploration and constraint checking.

Conclusion

The Water Jug and the Missionaries & Cannibals experiments are examples of Artificial Intelligence basics. They show us how to represent states satisfy constraints and solve problems in a way. The Water Jug experiment is useful for understanding how the system changes when we do things to it. We can see how the states change and what happens when we make moves.

The Missionaries & Cannibals experiment is good for learning how to think and find our way back when we make a wrong move. It also teaches us how to move from one state to another without breaking any rules.